

Solving and Generating Sudoku as a CSP

A constraint-aware solver, generator, and interactive demonstration

Arjun Mohunta · Haoyang Ding · Rajat Choudhary

5	1 2 3 6 7 8 9	7	6	1 2 3 8 9	1 2 8 9	9	1 2 3 4 6 8	1 2 3 4 6 8
1 2 3 4 8 9	8	1 2 3 4 6 9	1 2 4 6 7 9	9	1 2 3 4 6 7	1 2 4 6 7 8	5	1 2 3 7 9
1 2 4 5 9	1 2 3 6 9	9	1 2 3 4 5 8	2 3 5 8	7	1 2 4 6 8	1 2 3 4 8	1 2 3 4 5 8
6	1 2 3 4 7 8	1 2 4 5 8	8	1 2 3 5 7 9	1 2 5 6 9	2 3 4 7 9	1 2 3 4 5 9	7
1 2 3 7 8 9	1 2 6 9	1 2 3 5 6 7	1 2 3 7 9	4	2	1 2 3 6 8 9	1 2 6 8 9	1 3 6 8 9
1 2 3 7 8	1 2 3 5 8 9	4	2 3 6 7 9	1 2 7 9	5	1 2 3 6 8 9	1 2 3 6 7 9	1
7	1 2 4 6 8 9	1 2 5 6 8	1 2 3 6 9	1	1 2 3 4 8 9	2 3 4 5 9	8	1 2 6 9
1 2 3 6 9	6	1 2 5 8 9	1 2 4 7 9	2 3 6 8 9	1 2 3 7 9	3	1 2 4 5 7 9	1 2 5
1 2 4 5 8 9	1 2 3 4 8	1 2 6 7 8 9	3	2 3 4 6 8	8	1 2 4 5 9	1 2 4 6 7 9	2

Sudoku becomes a search problem when every blank has nine possibilities

The rules are local. The search space is not.

Model each cell as a variable with domain 1–9, then enforce all-different constraints across every row, column, and box.

81

cell variables

27

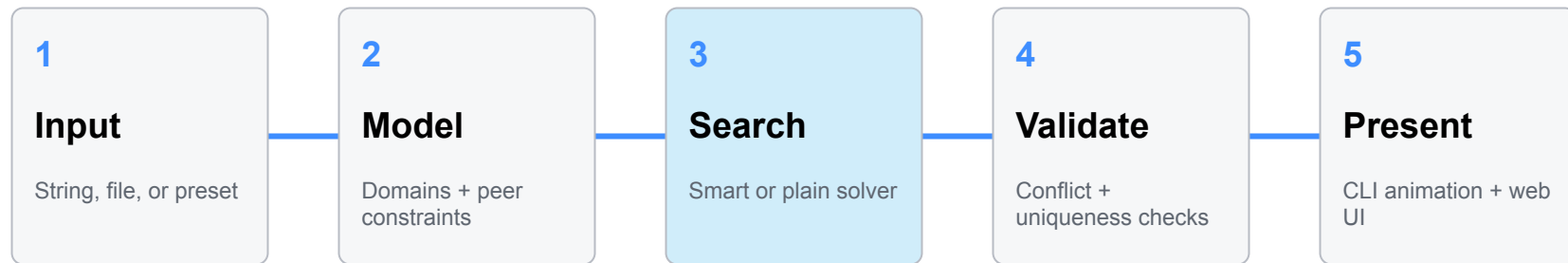
all-different units

20

unique peers per cell

Without informed ordering, the solver repeatedly explores choices the constraints could have ruled out earlier.

One CSP core powers solving, generation, and two demonstrations



Generator loop: complete grid → remove clue → keep only if exactly one solution remains

MRV and forward checking shrink the search before we guess

01

Initialize domains

Nine-bit masks remove candidates already used by peers.

02

Select with MRV

Choose the empty cell with the fewest remaining values.

03

Assign and prune

Update 20 peers; undo immediately when a domain becomes empty.

Same backtracking framework. Better variable order. Earlier contradictions.

Baseline: row-major backtracking does the least-informed search

```
$ python3 demo.py compare --no-color
```

puzzle	clues	smart bt	plain bt
easy	30	0	4,157
escargot	23	151	8,911
extreme	17	4,048	>400,000

plain search: first empty cell, values 1-9
node cap: 400,000

What the baseline lacks

- No candidate domains
- No MRV variable ordering
- No forward propagation
- **Safe node cap prevents hangs**

A useful baseline is simple enough that every improvement has a clear cause.

Best run: the 17-clue extreme puzzle finishes in about 9 ms

17 clues / INPUT

```
. . . | . . . | . 1 .  
4 . . | . . . | . . .  
. 2 . | . . . | . . .  
-----+-----+-----  
. . . | . 5 . | 4 . 7  
. . 8 | . . . | 3 . .  
. . 1 | . 9 . | . . .  
-----+-----+-----  
3 . . | 4 . . | 2 . .  
. 5 . | 1 . . | . . .  
. . . | 8 . 6 | . . .
```

UNIQUE SOLUTION / OUTPUT

```
6 9 3 | 7 8 4 | 5 1 2  
4 8 7 | 5 1 2 | 9 3 6  
1 2 5 | 9 6 3 | 8 7 4  
-----+-----+-----  
9 3 2 | 6 5 1 | 4 8 7  
5 6 8 | 2 4 7 | 3 9 1  
7 4 1 | 3 9 8 | 6 2 5  
-----+-----+-----  
3 1 9 | 4 7 5 | 2 6 8  
8 5 6 | 1 2 9 | 7 4 3  
2 7 4 | 8 3 6 | 1 5 9
```

9.087

median ms

4,113

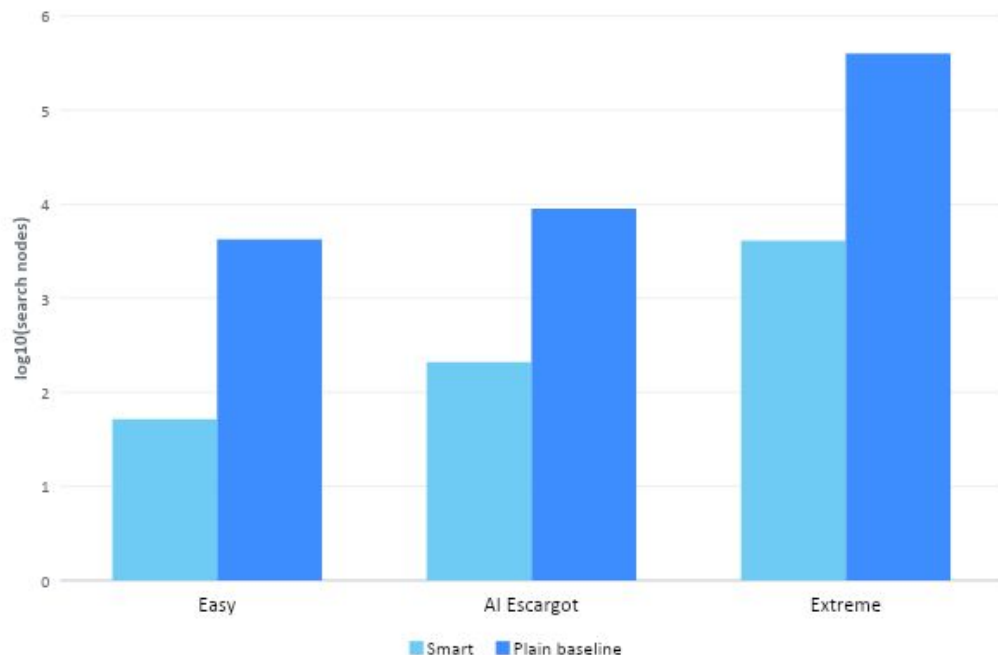
search nodes

1

unique solution

`demo.py solve extreme`

The smart solver cuts search by one to two orders of magnitude



81×

fewer nodes on Easy

43×

fewer nodes on AI Escargot

>97×

reduction before baseline cap

Extreme baseline stopped at 400,001 nodes; its ratio is a lower bound.

Generated difficulty follows clue count—not guaranteed human difficulty

30 generated puzzles per level · fixed seeds 0–29

Level	Clues	Zero-backtrack	Median backtracks	Observed range
Easy	45	30 / 30	0	0–0
Medium	34	27 / 30	0	0–12
Hard	26	4 / 30	32.5	0–277

Clue count separates the targets, but solver effort still overlaps—some 26-clue puzzles need zero backtracking.

The experiments also expose four important limitations

- | | | |
|----|---|---|
| 01 | Difficulty is a proxy | Clue count does not guarantee human difficulty; generated levels overlap. |
| 02 | Propagation is intentionally basic | No advanced human strategies or stronger consistency algorithms. |
| 03 | Traces can grow | Python records the full animation trace; the web demo caps events for safety. |
| 04 | Evidence is local | 50 timing repeats and 30 seeds per level; timing remains hardware-dependent. |

Uniqueness is guaranteed; minimality and technique-based grading are not.

Constraint-aware search is the difference between a demo and a dead end.

Formalize constraints · Choose the most constrained cell · Measure against a baseline

Questions?